

股票 Alpha 因子发掘系统 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (- []) syntax for tracking.

Goal: 构建一套巴菲特风格、面向 S&P 500、月度调仓的价值投资 alpha 因子发掘与回测系统，CLI 输出排名股票清单与回测报告。

Architecture: 轻量模块化管线。6 个单一职责模块（data / factors / evaluation / model / backtest / cli），层间用带 `MultiIndex(date, ticker)` 的 pandas DataFrame + parquet 通信。data 层屏蔽数据源差异（原型用 yfinance），factors 层为纯函数，model 层用 LightGBM walk-forward 合成打分，backtest 层自建月频调仓引擎。

Tech Stack: Python 3.11+, pandas, numpy, yfinance, lightgbm, alphasens-reloaded, pyfolio-reloaded, pyarrow, loguru, typer, pytest.

File Structure

```
alpha-studio/
├── pyproject.toml
├── requirements.txt
├── README.md
├── src/alpha_studio/
│   ├── __init__.py
│   ├── config.py                # 路径、常量（缓存目录、调仓频率、成本）
│   ├── data/
│   │   ├── __init__.py
│   │   ├── universe.py         # S&P 500 成分股列表
│   │   ├── prices.py           # yfinance 日线 + parquet 缓存
│   │   └── fundamentals.py     # yfinance 基本面 + parquet 缓存
│   ├── factors/
│   │   ├── __init__.py
│   │   ├── definitions.py      # 各因子纯函数
│   │   ├── lag.py              # point-in-time 财报滞后
│   │   └── pipeline.py         # 组装横截面因子矩阵 + 标准化
│   ├── evaluation/
│   │   ├── __init__.py
│   │   └── alphasens_eval.py   # Alphasens 封装
│   ├── model/
│   │   ├── __init__.py
│   │   └── scorer.py           # LightGBM walk-forward 合成打分
│   ├── backtest/
│   │   ├── __init__.py
│   │   ├── engine.py           # 月频调仓回测
│   │   └── report.py           # pyfolio tearsheet + 指标
│   └── cli/
│       └── __init__.py
```

```
├── main.py          # typer 命令
└── tests/
    ├── __init__.py
    ├── conftest.py    # 共享 fixtures (构造小型数据)
    ├── data/test_universe.py
    ├── data/test_prices.py
    ├── data/test_fundamentals.py
    ├── factors/test_definitions.py
    ├── factors/test_lag.py
    ├── factors/test_pipeline.py
    ├── model/test_scorer.py
    └── backtest/test_engine.py
```

Task 1: 项目脚手架

Files:

- Create: `requirements.txt`
- Create: `pyproject.toml`
- Create: `src/alpha_studio/__init__.py`
- Create: `src/alpha_studio/config.py`
- Create: `tests/__init__.py`
- Create: `tests/conftest.py`
- ☐ **Step 1: 创建 requirements.txt**

```
pandas>=2.0
numpy>=1.24
pyarrow>=14
yfinance>=0.2.40
lightgbm>=4.0
alphalens-reloaded>=0.4.3
pyfolio-reloaded>=0.9.5
loguru>=0.7
typer>=0.12
scikit-learn>=1.3
pytest>=8.0
```

- ☐ **Step 2: 创建 pyproject.toml**

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
```

```
[project]
name = "alpha-studio"
version = "0.1.0"
requires-python = ">=3.11"
dynamic = ["dependencies"]

[tool.setuptools.dynamic]
dependencies = { file = ["requirements.txt"] }

[tool.setuptools.packages.find]
where = ["src"]

[project.scripts]
sp = "alpha_studio.cli.main:app"

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

- ❑ **Step 3: 创建包入口与 config**

src/alpha_studio/__init__.py:

```
__version__ = "0.1.0"
```

src/alpha_studio/config.py:

```
from pathlib import Path

ROOT = Path(__file__).resolve().parents[2]
CACHE_DIR = ROOT / "data_cache"
PRICES_DIR = CACHE_DIR / "prices"
FUNDAMENTALS_DIR = CACHE_DIR / "fundamentals"
REPORTS_DIR = ROOT / "reports"

# 回测/策略默认参数
REBALANCE_FREQ = "ME"          # pandas month-end
TOP_N = 25                     # 每期持仓数
TRANSACTION_COST = 0.001       # 单边成本（手续费+滑点）
FUNDAMENTAL_LAG_DAYS = 60      # 财报发布滞后天数
BENCHMARK = "^GSPC"           # S&P 500 指数

for _d in (CACHE_DIR, PRICES_DIR, FUNDAMENTALS_DIR, REPORTS_DIR):
    _d.mkdir(parents=True, exist_ok=True)
```

- ❑ **Step 4: 创建空 tests 包与 conftest**

tests/__init__.py: (空文件)

tests/conftest.py:

```
import numpy as np
import pandas as pd
import pytest

@pytest.fixture
def sample_fundamentals():
    """两只股票、两个财报期的小型基本面数据。"""
    idx = pd.MultiIndex.from_tuples(
        [("2023-03-31", "AAA"), ("2023-03-31", "BBB"),
         ("2023-06-30", "AAA"), ("2023-06-30", "BBB")],
        names=["report_date", "ticker"],
    )
    return pd.DataFrame(
        {
            "net_income": [100.0, 50.0, 110.0, 40.0],
            "total_equity": [1000.0, 1000.0, 1000.0, 1000.0],
            "total_assets": [2000.0, 2500.0, 2000.0, 2500.0],
            "revenue": [500.0, 300.0, 520.0, 280.0],
            "gross_profit": [200.0, 90.0, 210.0, 84.0],
            "total_debt": [500.0, 1500.0, 500.0, 1500.0],
            "free_cash_flow": [80.0, 20.0, 85.0, 15.0],
            "market_cap": [5000.0, 2000.0, 5000.0, 2000.0],
            "shares_out": [100.0, 100.0, 100.0, 100.0],
        },
        index=idx,
    )

@pytest.fixture
def sample_prices():
    """两只股票 4 个月末的收盘价（用于回测）。"""
    dates = pd.to_datetime(["2023-01-31", "2023-02-28", "2023-03-31", "2023-04-30"])
    idx = pd.MultiIndex.from_product([dates, ["AAA", "BBB"]], names=["date", "ticker"])
    # AAA 稳定上涨, BBB 下跌
    close = [10, 20, 11, 19, 12, 18, 13, 17]
    return pd.DataFrame({"close": close}, index=idx, dtype=float)
```

- ❑ Step 5: 安装依赖并验证 pytest 可运行

Run: `cd C:\Users\liti\alpha-studio; python -m pip install -e .` Expected: 安装成功
(lightgbm/pyfolio 可能需较长时间)

Run: `cd C:\Users\liti\alpha-studio; python -m pytest -q` Expected: no tests ran (无测试, 退出码 5), 证明 pytest 配置正确

- ❑ Step 6: Commit

```
git add -A
git commit -m "chore: project scaffold, config, pytest setup"
```

Task 2: Universe — S&P 500 成分股

Files:

- Create: `src/alpha_studio/data/__init__.py` (空)
- Create: `src/alpha_studio/data/universe.py`
- Create: `tests/data/__init__.py` (空)
- Create: `tests/data/test_universe.py`
- ☐ **Step 1: Write the failing test**

`tests/data/test_universe.py`:

```
from unittest.mock import patch
import pandas as pd
from alpha_studio.data import universe

def test_get_sp500_tickers_parses_wikipedia_table():
    fake_table = pd.DataFrame({"Symbol": ["AAPL", "MSFT", "BRK.B"]})
    with patch("alpha_studio.data.universe.pd.read_html", return_value=
[fake_table]):
        tickers = universe.get_sp500_tickers()
        assert "AAPL" in tickers
        assert "MSFT" in tickers
        # yfinance 用 '-' 而非 '.'
        assert "BRK-B" in tickers
        assert "BRK.B" not in tickers
```

- ☐ **Step 2: Run test to verify it fails**

Run: `python -m pytest tests/data/test_universe.py -v` Expected: FAIL — `ModuleNotFoundError: alpha_studio.data.universe`

- ☐ **Step 3: Write minimal implementation**

`src/alpha_studio/data/__init__.py`: (空文件)

`src/alpha_studio/data/universe.py`:

```
import pandas as pd
```

```
SP500_WIKI_URL = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
```

```
def get_sp500_tickers() -> list[str]:
    """从 Wikipedia 抓取当前 S&P 500 成分股, 返回 yfinance 风格 ticker。"""
    tables = pd.read_html(SP500_WIKI_URL)
    symbols = tables[0]["Symbol"].astype(str).tolist()
    return [s.replace(".", "-").strip() for s in symbols]
```

- ☐ Step 4: Run test to verify it passes

Run: `python -m pytest tests/data/test_universe.py -v` Expected: PASS

- ☐ Step 5: Commit

```
git add -A
git commit -m "feat(data): S&P 500 universe loader"
```

Task 3: 价格数据采集 + 缓存

Files:

- Create: `src/alpha_studio/data/prices.py`
- Create: `tests/data/test_prices.py`
- ☐ Step 1: Write the failing test

`tests/data/test_prices.py`:

```
from unittest.mock import patch
import pandas as pd
from alpha_studio.data import prices

def _fake_yf_download(tickers, **kwargs):
    dates = pd.to_datetime(["2023-01-31", "2023-02-28"])
    cols = pd.MultiIndex.from_product([["Close"], tickers])
    data = [[10.0, 100.0], [11.0, 110.0]]
    return pd.DataFrame(data, index=dates, columns=cols)

def test_fetch_prices_returns_long_multiindex():
    with patch("alpha_studio.data.prices.yf.download",
side_effect=_fake_yf_download):
        df = prices.fetch_prices(["AAA", "BBB"], "2023-01-01", "2023-03-01",
use_cache=False)
        assert list(df.index.names) == ["date", "ticker"]
        assert "close" in df.columns
```

```
# 长表: 2 日 * 2 股 = 4 行
assert len(df) == 4
assert df.loc[(pd.Timestamp("2023-01-31"), "AAA"), "close"] == 10.0
```

- ☐ Step 2: Run test to verify it fails

Run: `python -m pytest tests/data/test_prices.py -v` Expected: FAIL — `ModuleNotFoundError: alpha_studio.data.prices`

- ☐ Step 3: Write minimal implementation

`src/alpha_studio/data/prices.py`:

```
import pandas as pd
import yfinance as yf
from loguru import logger

from alpha_studio.config import PRICES_DIR

def _cache_path(start: str, end: str):
    return PRICES_DIR / f"prices_{start}_{end}.parquet"

def fetch_prices(tickers: list[str], start: str, end: str, use_cache: bool = True)
-> pd.DataFrame:
    """拉取日线收盘价, 返回 MultiIndex(date, ticker) 长表, 列含 'close'。"""
    cache = _cache_path(start, end)
    if use_cache and cache.exists():
        logger.info(f"price cache hit: {cache}")
        return pd.read_parquet(cache)

    raw = yf.download(tickers, start=start, end=end, auto_adjust=True,
progress=False)
    close = raw["Close"]
    if isinstance(close, pd.Series): # 单只股票
        close = close.to_frame(tickers[0])
    long = (
        close.stack(future_stack=True)
        .rename("close")
        .to_frame()
    )
    long.index = long.index.set_names(["date", "ticker"])
    long = long.dropna()

    if use_cache:
        long.to_parquet(cache)
    return long
```

- ☐ Step 4: Run test to verify it passes

Run: `python -m pytest tests/data/test_prices.py -v` Expected: PASS

- ❑ **Step 5: Commit**

```
git add -A
git commit -m "feat(data): yfinance price fetcher with parquet cache"
```

Task 4: 基本面数据采集 + 缓存

Files:

- Create: `src/alpha_studio/data/fundamentals.py`
- Create: `tests/data/test_fundamentals.py`

财报字段从 yfinance 的 `Ticker.quarterly_financials / quarterly_balance_sheet / quarterly_cashflow` 提取。本任务封装"原始 yfinance 报表 → 标准化字段表"的转换，便于单测且屏蔽数据源。

- ❑ **Step 1: Write the failing test**

`tests/data/test_fundamentals.py`:

```
import pandas as pd
from alpha_studio.data import fundamentals

def test_normalize_statements_maps_fields():
    # 模拟 yfinance: 行是科目, 列是报告期
    periods = pd.to_datetime(["2023-03-31", "2023-06-30"])
    income = pd.DataFrame(
        {periods[0]: [100.0, 500.0, 200.0], periods[1]: [110.0, 520.0, 210.0]},
        index=["Net Income", "Total Revenue", "Gross Profit"],
    )
    balance = pd.DataFrame(
        {periods[0]: [1000.0, 2000.0, 500.0], periods[1]: [1000.0, 2000.0,
500.0]},
        index=["Stockholders Equity", "Total Assets", "Total Debt"],
    )
    cashflow = pd.DataFrame(
        {periods[0]: [80.0], periods[1]: [85.0]},
        index=["Free Cash Flow"],
    )
    out = fundamentals.normalize_statements("AAA", income, balance, cashflow)
    assert list(out.index.names) == ["report_date", "ticker"]
    row = out.loc[(pd.Timestamp("2023-03-31"), "AAA")]
    assert row["net_income"] == 100.0
    assert row["total_equity"] == 1000.0
```

```
assert row["revenue"] == 500.0
assert row["free_cash_flow"] == 80.0
```

- ☐ **Step 2: Run test to verify it fails**

Run: `python -m pytest tests/data/test_fundamentals.py -v` Expected: FAIL —
 ModuleNotFoundError: alpha_studio.data.fundamentals

- ☐ **Step 3: Write minimal implementation**

src/alpha_studio/data/fundamentals.py:

```
import pandas as pd
import yfinance as yf
from loguru import logger

from alpha_studio.config import FUNDAMENTALS_DIR

# yfinance 报表行名 → 标准化字段名
FIELD_MAP = {
    "Net Income": "net_income",
    "Total Revenue": "revenue",
    "Gross Profit": "gross_profit",
    "Stockholders Equity": "total_equity",
    "Total Assets": "total_assets",
    "Total Debt": "total_debt",
    "Free Cash Flow": "free_cash_flow",
}

def _extract(statement: pd.DataFrame, wanted: dict) -> pd.DataFrame:
    rows = {}
    for raw_name, std_name in wanted.items():
        if raw_name in statement.index:
            rows[std_name] = statement.loc[raw_name]
    if not rows:
        return pd.DataFrame()
    return pd.DataFrame(rows) # index = report periods

def normalize_statements(ticker, income, balance, cashflow) -> pd.DataFrame:
    """把三张 yfinance 报表合并为标准化字段的长表。"""
    parts = [_extract(s, FIELD_MAP) for s in (income, balance, cashflow)]
    merged = pd.concat(parts, axis=1)
    merged.index = pd.to_datetime(merged.index)
    merged.index.name = "report_date"
    merged["ticker"] = ticker
    merged = merged.set_index("ticker", append=True)
    return merged.sort_index()

def fetch_fundamentals(tickers: list[str], use_cache: bool = True) ->
```

```

pd.DataFrame:
    """逐只拉取季度财报，合并为标准化长表。单只失败则跳过。"""
    cache = FUNDAMENTALS_DIR / "fundamentals.parquet"
    if use_cache and cache.exists():
        logger.info(f"fundamentals cache hit: {cache}")
        return pd.read_parquet(cache)

    frames = []
    for t in tickers:
        try:
            tk = yf.Ticker(t)
            norm = normalize_statements(
                t, tk.quarterly_financials, tk.quarterly_balance_sheet,
                tk.quarterly_cashflow
            )
            # market_cap / shares_out 取最新快照
            info = tk.fast_info
            norm["market_cap"] = getattr(info, "market_cap", None)
            norm["shares_out"] = getattr(info, "shares", None)
            frames.append(norm)
        except Exception as e: # noqa: BLE001
            logger.warning(f"skip {t}: {e}")
            continue

    if not frames:
        return pd.DataFrame()
    out = pd.concat(frames).sort_index()
    if use_cache:
        out.to_parquet(cache)
    return out

```

- ☐ Step 4: Run test to verify it passes

Run: `python -m pytest tests/data/test_fundamentals.py -v` Expected: PASS

- ☐ Step 5: Commit

```

git add -A
git commit -m "feat(data): yfinance fundamentals normalizer + fetcher"

```

Task 5: 因子定义（纯函数）

Files:

- Create: `src/alpha_studio/factors/__init__.py` (空)
- Create: `src/alpha_studio/factors/definitions.py`
- Create: `tests/factors/__init__.py` (空)

- Create: `tests/factors/test_definitions.py`

- ☐ **Step 1: Write the failing test**

`tests/factors/test_definitions.py`:

```
import numpy as np
from alpha_studio.factors import definitions as fd

def test_compute_factors_known_values(sample_fundamentals):
    out = fd.compute_factors(sample_fundamentals)
    # AAA 2023-03-31: ROE = net_income/total_equity = 100/1000 = 0.1
    aaa = out.loc[("2023-03-31", "AAA")]
    assert np.isclose(aaa["roe"], 0.1)
    # ROA = 100/2000 = 0.05
    assert np.isclose(aaa["roa"], 0.05)
    # gross_margin = 200/500 = 0.4
    assert np.isclose(aaa["gross_margin"], 0.4)
    # debt_to_equity = 500/1000 = 0.5
    assert np.isclose(aaa["debt_to_equity"], 0.5)
    # fcf_yield = free_cash_flow/market_cap = 80/5000 = 0.016
    assert np.isclose(aaa["fcf_yield"], 0.016)
    # pb = market_cap/total_equity = 5000/1000 = 5.0
    assert np.isclose(aaa["pb"], 5.0)
    # earnings_yield = net_income/market_cap = 100/5000 = 0.02 (PE 倒数)
    assert np.isclose(aaa["earnings_yield"], 0.02)

def test_compute_factors_handles_zero_denominator():
    import pandas as pd
    idx = pd.MultiIndex.from_tuples([("2023-03-31", "ZZZ")], names=["report_date",
"ticker"])
    df = pd.DataFrame(
        {"net_income": [10.0], "total_equity": [0.0], "total_assets": [100.0],
        "revenue": [0.0], "gross_profit": [0.0], "total_debt": [0.0],
        "free_cash_flow": [5.0], "market_cap": [100.0], "shares_out": [10.0]},
        index=idx,
    )
    out = fd.compute_factors(df)
    # 除零应得 NaN 而非 inf
    assert np.isnan(out.loc[("2023-03-31", "ZZZ"), "roe"])
```

- ☐ **Step 2: Run test to verify it fails**

Run: `python -m pytest tests/factors/test_definitions.py -v` Expected: FAIL —
ModuleNotFoundError: `alpha_studio.factors.definitions`

- ☐ **Step 3: Write minimal implementation**

`src/alpha_studio/factors/__init__.py`: (空文件)

src/alpha_studio/factors/definitions.py:

```
import numpy as np
import pandas as pd

def _safe_div(a: pd.Series, b: pd.Series) -> pd.Series:
    """除法，分母为 0 或缺失返回 NaN（不产生 inf）。"""
    b = b.replace(0, np.nan)
    return a / b

def compute_factors(fundamentals: pd.DataFrame) -> pd.DataFrame:
    """从标准化基本面字段计算价值/质量因子。输入输出均 MultiIndex(report_date,
    ticker)。"""
    f = fundamentals
    out = pd.DataFrame(index=f.index)
    # 盈利能力
    out["roe"] = _safe_div(f["net_income"], f["total_equity"])
    out["roa"] = _safe_div(f["net_income"], f["total_assets"])
    out["gross_margin"] = _safe_div(f["gross_profit"], f["revenue"])
    out["net_margin"] = _safe_div(f["net_income"], f["revenue"])
    # 估值（用 yield 形式，越大越便宜，方向统一）
    out["earnings_yield"] = _safe_div(f["net_income"], f["market_cap"])
    out["fcf_yield"] = _safe_div(f["free_cash_flow"], f["market_cap"])
    out["pb"] = _safe_div(f["market_cap"], f["total_equity"])
    out["book_to_market"] = _safe_div(f["total_equity"], f["market_cap"])
    # 质量
    out["debt_to_equity"] = _safe_div(f["total_debt"], f["total_equity"])
    return out

# 进入模型的因子方向：True 表示值越大越看好
FACTOR_DIRECTION = {
    "roe": True, "roa": True, "gross_margin": True, "net_margin": True,
    "earnings_yield": True, "fcf_yield": True, "book_to_market": True,
    "pb": False, "debt_to_equity": False,
}
```

- ☐ Step 4: Run test to verify it passes

Run: `python -m pytest tests/factors/test_definitions.py -v` Expected: PASS（两个测试均通过）

- ☐ Step 5: Commit

```
git add -A
git commit -m "feat(factors): value/quality factor definitions"
```

Task 6: Point-in-time 财报滞后

Files:

- Create: `src/alpha_studio/factors/lag.py`
- Create: `tests/factors/test_lag.py`

防未来函数核心：财报期 `report_date` 不能在该日立即使用，须延后 `FUNDAMENTAL_LAG_DAYS` 天才算“可见”。本模块把因子从 `report_date` 对齐到月末调仓日（forward-fill 最近可见财报）。

- ☐ **Step 1: Write the failing test**

`tests/factors/test_lag.py`:

```
import pandas as pd
from alpha_studio.factors import lag

def test_lag_factors_no_lookahead():
    # AAA 在 2023-03-31 发布财报，滞后 60 天 → 2023-05-30 才可见
    idx = pd.MultiIndex.from_tuples(
        [("2023-03-31", "AAA"), ("2023-06-30", "AAA")],
        names=["report_date", "ticker"],
    )
    factors = pd.DataFrame({"roe": [0.1, 0.2]}, index=idx)
    rebal_dates = pd.to_datetime(["2023-04-30", "2023-05-31", "2023-06-30"])

    aligned = lag.align_to_rebalance(factors, rebal_dates, lag_days=60)

    # 2023-04-30: 财报尚不可见 (3-31 + 60 = 5-30) → 无数据
    assert ("2023-04-30", "AAA") not in aligned.index
    # 2023-05-31: 3-31 财报已可见 → roe = 0.1
    assert aligned.loc[(pd.Timestamp("2023-05-31"), "AAA"), "roe"] == 0.1
    # 2023-06-30: 6-30 财报刚发布尚未可见，仍用 3-31 的 0.1
    assert aligned.loc[(pd.Timestamp("2023-06-30"), "AAA"), "roe"] == 0.1
```

- ☐ **Step 2: Run test to verify it fails**

Run: `python -m pytest tests/factors/test_lag.py -v` Expected: FAIL — `ModuleNotFoundError: alpha_studio.factors.lag`

- ☐ **Step 3: Write minimal implementation**

`src/alpha_studio/factors/lag.py`:

```
import pandas as pd

def align_to_rebalance(factors: pd.DataFrame, rebalance_dates, lag_days: int) ->
pd.DataFrame:
    """把按 report_date 的因子对齐到调仓日，财报延后 lag_days 才可见（无未来函数）。
```

```

对每个调仓日，取该日前已可见（report_date + lag_days <= 调仓日）的最近一期财报。
返回 MultiIndex(date, ticker)。
"""
f = factors.reset_index()
f["available_date"] = f["report_date"] + pd.Timedelta(days=lag_days)
rebalance_dates = pd.DatetimeIndex(rebalance_dates)

rows = []
for ticker, grp in f.groupby("ticker"):
    grp = grp.sort_values("available_date")
    for d in rebalance_dates:
        visible = grp[grp["available_date"] <= d]
        if visible.empty:
            continue
        latest = visible.iloc[-1]
        rec = latest.drop(["report_date", "available_date",
"ticker"]).to_dict()
        rec["date"] = d
        rec["ticker"] = ticker
        rows.append(rec)

if not rows:
    return pd.DataFrame()
out = pd.DataFrame(rows).set_index(["date", "ticker"]).sort_index()
return out

```

- ☐ **Step 4: Run test to verify it passes**

Run: `python -m pytest tests/factors/test_lag.py -v` Expected: PASS

- ☐ **Step 5: Commit**

```

git add -A
git commit -m "feat(factors): point-in-time fundamental lag alignment"

```

Task 7: 因子管线（横截面标准化 + 合成）

Files:

- Create: `src/alpha_studio/factors/pipeline.py`
- Create: `tests/factors/test_pipeline.py`
- ☐ **Step 1: Write the failing test**

`tests/factors/test_pipeline.py`:

```

import numpy as np
import pandas as pd
from alpha_studio.factors import pipeline

def test_cross_sectional_zscore_per_date():
    idx = pd.MultiIndex.from_tuples(
        [("2023-05-31", "AAA"), ("2023-05-31", "BBB"), ("2023-05-31", "CCC")],
        names=["date", "ticker"],
    )
    df = pd.DataFrame({"roe": [0.1, 0.2, 0.3], "pb": [5.0, 3.0, 1.0]}, index=idx)
    out = pipeline.cross_sectional_zscore(df)
    # 每个调仓日内均值≈0
    day = out.loc["2023-05-31"]
    assert np.isclose(day["roe"].mean(), 0.0, atol=1e-9)
    assert np.isclose(day["roe"].std(ddof=0), 1.0, atol=1e-9)

def test_apply_direction_flips_negative_factors():
    idx = pd.MultiIndex.from_tuples(
        [("2023-05-31", "AAA"), ("2023-05-31", "BBB")],
        names=["date", "ticker"],
    )
    # pb 方向为 False (越小越好), 应被翻转
    z = pd.DataFrame({"roe": [1.0, -1.0], "pb": [1.0, -1.0]}, index=idx)
    out = pipeline.apply_direction(z, {"roe": True, "pb": False})
    assert out.loc[("2023-05-31", "AAA"), "pb"] == -1.0 # 翻转后高 pb 变低分
    assert out.loc[("2023-05-31", "AAA"), "roe"] == 1.0

```

- ☐ Step 2: Run test to verify it fails

Run: `python -m pytest tests/factors/test_pipeline.py -v` Expected: FAIL —

`ModuleNotFoundError: alpha_studio.factors.pipeline`

- ☐ Step 3: Write minimal implementation

`src/alpha_studio/factors/pipeline.py`:

```

import pandas as pd

from alpha_studio.data.prices import fetch_prices
from alpha_studio.data.fundamentals import fetch_fundamentals
from alpha_studio.factors.definitions import compute_factors, FACTOR_DIRECTION
from alpha_studio.factors.lag import align_to_rebalance
from alpha_studio.config import FUNDAMENTAL_LAG_DAYS

def cross_sectional_zscore(factors: pd.DataFrame) -> pd.DataFrame:
    """对每个调仓日(date)内做横截面 z-score 标准化。"""
    def _z(group):

```

```

        return (group - group.mean()) / group.std(ddof=0)
    return factors.groupby(level="date", group_keys=False).apply(_z)

def apply_direction(zscores: pd.DataFrame, direction: dict) -> pd.DataFrame:
    """按因子方向翻转：方向为 False 的因子乘 -1，使所有因子'越大越好'。"""
    out = zscores.copy()
    for col, positive in direction.items():
        if col in out.columns and not positive:
            out[col] = -out[col]
    return out

def build_factor_matrix(factors: pd.DataFrame) -> pd.DataFrame:
    """标准化 + 方向归一，输出可直接喂模型的因子矩阵。"""
    z = cross_sectional_zscore(factors)
    return apply_direction(z, FACTOR_DIRECTION)

def build_from_raw(tickers, start, end, rebalance_dates, use_cache=True) ->
pd.DataFrame:
    """端到端：拉基本面→算因子→滞后对齐→标准化。返回 MultiIndex(date, ticker) 因子矩
    阵。"""
    fund = fetch_fundamentals(tickers, use_cache=use_cache)
    raw_factors = compute_factors(fund)
    aligned = align_to_rebalance(raw_factors, rebalance_dates,
    FUNDAMENTAL_LAG_DAYS)
    return build_factor_matrix(aligned)

```

- ☐ Step 4: Run test to verify it passes

Run: `python -m pytest tests/factors/test_pipeline.py -v` Expected: PASS (两个测试均通过)

- ☐ Step 5: Commit

```

git add -A
git commit -m "feat(factors): cross-sectional standardization pipeline"

```

Task 8: 模型 — LightGBM walk-forward 打分

Files:

- Create: `src/alpha_studio/model/__init__.py` (空)
- Create: `src/alpha_studio/model/scorer.py`
- Create: `tests/model/__init__.py` (空)
- Create: `tests/model/test_scorer.py`

模型用历史 (因子 → 未来 1 月收益) 训练 LightGBM，对当期横截面输出综合打分。严格 walk-forward：预测某调仓日时只用该日之前的样本。

- ❑ **Step 1: Write the failing test**

tests/model/test_scorer.py:

```
import numpy as np
import pandas as pd
from alpha_studio.model import scorer

def _make_dataset(n_dates=8, n_stocks=20, seed=0):
    rng = np.random.default_rng(seed)
    dates = pd.date_range("2022-01-31", periods=n_dates, freq="ME")
    tickers = [f"S{i:02d}" for i in range(n_stocks)]
    idx = pd.MultiIndex.from_product([dates, tickers], names=["date", "ticker"])
    roe = rng.normal(size=len(idx))
    pb = rng.normal(size=len(idx))
    factors = pd.DataFrame({"roe": roe, "pb": pb}, index=idx)
    # 未来收益与 roe 正相关, 便于验证模型学到信号
    fwd = 0.05 * factors["roe"] + rng.normal(scale=0.01, size=len(idx))
    return factors, fwd.rename("fwd_return")

def test_compute_forward_returns_shifts_back():
    # 价格已知, 未来收益 = 下一调仓日收益
    dates = pd.to_datetime(["2023-01-31", "2023-02-28", "2023-03-31"])
    idx = pd.MultiIndex.from_product([dates, ["AAA"]], names=["date", "ticker"])
    prices = pd.DataFrame({"close": [10.0, 11.0, 11.0]}, index=idx)
    fwd = scorer.compute_forward_returns(prices)
    # 2023-01-31 的未来收益 = 11/10 - 1 = 0.1
    assert np.isclose(fwd.loc[(pd.Timestamp("2023-01-31"), "AAA")], 0.1)
    # 最后一期无未来收益
    assert (pd.Timestamp("2023-03-31"), "AAA") not in fwd.dropna().index

def test_walk_forward_scores_align_with_signal():
    factors, fwd = _make_dataset()
    scores = scorer.walk_forward_score(factors, fwd, min_train_dates=4)
    # 只在第 5 个调仓日起才有打分 (前 4 期用于训练)
    scored_dates = scores.index.get_level_values("date").unique()
    assert len(scored_dates) == 4
    # 在最后一个调仓日, roe 高的股票打分应整体更高 (rank 相关为正)
    last = scores.index.get_level_values("date").max()
    merged = pd.concat([scores.xs(last, level="date"),
                        factors.xs(last, level="date")["roe"]], axis=1).dropna()
    corr = merged["score"].corr(merged["roe"], method="spearman")
    assert corr > 0.3
```

- ❑ **Step 2: Run test to verify it fails**

Run: `python -m pytest tests/model/test_scorer.py -v` Expected: FAIL — `ModuleNotFoundError: alpha_studio.model.scorer`

- **Step 3: Write minimal implementation**

src/alpha_studio/model/__init__.py: (空文件)

src/alpha_studio/model/scorer.py:

```
import lightgbm as lgb
import numpy as np
import pandas as pd

def compute_forward_returns(prices: pd.DataFrame) -> pd.Series:
    """每个调仓日的未来 1 期收益（下一调仓日 close / 当前 close - 1）。"""
    close = prices["close"].unstack("ticker").sort_index()
    fwd = close.shift(-1) / close - 1.0
    return fwd.stack(future_stack=True).rename("fwd_return")

def _train_predict(train_X, train_y, pred_X) -> np.ndarray:
    model = lgb.LGBMRegressor(
        n_estimators=200, max_depth=3, num_leaves=7,
        learning_rate=0.05, min_child_samples=20,
        subsample=0.8, colsample_bytree=0.8, verbose=-1,
    )
    model.fit(train_X, train_y)
    return model.predict(pred_X)

def walk_forward_score(factors: pd.DataFrame, fwd_returns: pd.Series,
                       min_train_dates: int = 12) -> pd.DataFrame:
    """walk-forward: 对每个调仓日，用之前所有（因子，未来收益）训练，预测当期打分。

    返回 MultiIndex(date, ticker)，单列 'score'。
    """
    data = factors.join(fwd_returns, how="inner")
    feature_cols = list(factors.columns)
    all_dates = sorted(data.index.get_level_values("date").unique())

    out_frames = []
    for i, d in enumerate(all_dates):
        if i < min_train_dates:
            continue
        train = data[data.index.get_level_values("date") < d].dropna(subset=
["fwd_return"])
        train = train.dropna(subset=feature_cols)
        pred = factors.xs(d, level="date",
drop_level=False).dropna(subset=feature_cols)
        if train.empty or pred.empty:
            continue
        preds = _train_predict(train[feature_cols], train["fwd_return"],
pred[feature_cols])
        out_frames.append(pd.DataFrame({"score": preds}, index=pred.index))
```

```
if not out_frames:
    return pd.DataFrame(columns=["score"])
return pd.concat(out_frames).sort_index()
```

- ☐ **Step 4: Run test to verify it passes**

Run: `python -m pytest tests/model/test_scorer.py -v` Expected: PASS (三个测试均通过; 信号相关性测试验证模型学到 roe 信号)

- ☐ **Step 5: Commit**

```
git add -A
git commit -m "feat(model): LightGBM walk-forward scorer"
```

Task 9: 回测引擎 (月频调仓 + 成本 + 指标)

Files:

- Create: `src/alpha_studio/backtest/__init__.py` (空)
- Create: `src/alpha_studio/backtest/engine.py`
- Create: `tests/backtest/__init__.py` (空)
- Create: `tests/backtest/test_engine.py`
- ☐ **Step 1: Write the failing test**

`tests/backtest/test_engine.py`:

```
import numpy as np
import pandas as pd
from alpha_studio.backtest import engine

def test_select_topn_picks_highest_scores():
    idx = pd.MultiIndex.from_tuples(
        [("2023-01-31", "AAA"), ("2023-01-31", "BBB"), ("2023-01-31", "CCC")],
        names=["date", "ticker"],
    )
    scores = pd.DataFrame({"score": [0.3, 0.1, 0.2]}, index=idx)
    holdings = engine.select_topn(scores, top_n=2)
    picked = holdings.xs("2023-01-31", level="date").index.tolist()
    assert set(picked) == {"AAA", "CCC"}
    # 等权
    assert np.isclose(holdings.iloc[0], 0.5)

def test_backtest_returns_match_equal_weight(sample_prices):
```

```

# 持仓：每期都持有 AAA + BBB 等权
dates = sample_prices.index.get_level_values("date").unique()[:-1]
idx = pd.MultiIndex.from_product([dates, ["AAA", "BBB"]], names=["date",
"ticker"])
weights = pd.Series(0.5, index=idx, name="weight")

result = engine.run_backtest(weights, sample_prices, cost=0.0)
# 第一期(1-31→2-28): AAA 10→11(+10%), BBB 20→19(-5%), 等权 = +2.5%
first_ret = result["returns"].iloc[0]
assert np.isclose(first_ret, 0.025)

def test_backtest_applies_transaction_cost(sample_prices):
    dates = sample_prices.index.get_level_values("date").unique()[:-1]
    idx = pd.MultiIndex.from_product([dates, ["AAA", "BBB"]], names=["date",
"ticker"])
    weights = pd.Series(0.5, index=idx, name="weight")

    no_cost = engine.run_backtest(weights, sample_prices, cost=0.0)
    ["returns"].iloc[0]
    with_cost = engine.run_backtest(weights, sample_prices, cost=0.01)
    ["returns"].iloc[0]
    # 首期建仓换手 100%, 成本拉低收益
    assert with_cost < no_cost

```

- ❑ **Step 2: Run test to verify it fails**

Run: `python -m pytest tests/backtest/test_engine.py -v` Expected: FAIL — `ModuleNotFoundError: alpha_studio.backtest.engine`

- ❑ **Step 3: Write minimal implementation**

`src/alpha_studio/backtest/__init__.py`: (空文件)

`src/alpha_studio/backtest/engine.py`:

```

import numpy as np
import pandas as pd

def select_topn(scores: pd.DataFrame, top_n: int) -> pd.Series:
    """每个调仓日取打分最高的 top_n 只，等权。返回 MultiIndex(date, ticker) 权重 Series。"""
    def _pick(group):
        top = group["score"].nlargest(top_n)
        w = pd.Series(1.0 / len(top), index=top.index)
        return w
    weights = scores.groupby(level="date", group_keys=False).apply(_pick)
    return weights.rename("weight")

def run_backtest(weights: pd.Series, prices: pd.DataFrame, cost: float) -> dict:

```

```

"""按权重逐期持有，计算扣成本后的组合收益序列与累计净值。

weights: MultiIndex(date, ticker) 各调仓日目标权重
prices: MultiIndex(date, ticker) 含 'close'
cost: 单边换手成本率
"""

close = prices["close"].unstack("ticker").sort_index()
period_ret = close.pct_change().shift(-1) # date 期初→下一日期的收益

w =
weights.unstack("ticker").reindex(columns=close.columns).fillna(0.0).sort_index()
rebal_dates = w.index

gross_returns = []
turnover_list = []
prev_w = pd.Series(0.0, index=close.columns)
for d in rebal_dates:
    cur_w = w.loc[d]
    turnover = (cur_w - prev_w).abs().sum()
    ret = (cur_w * period_ret.loc[d]).sum()
    net = ret - turnover * cost
    gross_returns.append(net)
    turnover_list.append(turnover)
    prev_w = cur_w

returns = pd.Series(gross_returns, index=rebal_dates, name="returns").dropna()
equity = (1.0 + returns).cumprod()
turnover = pd.Series(turnover_list, index=rebal_dates, name="turnover")
return {"returns": returns, "equity_curve": equity, "turnover": turnover}

```

- ☐ Step 4: Run test to verify it passes

Run: `python -m pytest tests/backtest/test_engine.py -v` Expected: PASS (三个测试均通过)

- ☐ Step 5: Commit

```

git add -A
git commit -m "feat(backtest): monthly rebalance engine with costs"

```

Task 10: 回测报告 + 因子评估封装

Files:

- Create: `src/alpha_studio/backtest/report.py`
- Create: `src/alpha_studio/evaluation/__init__.py` (空)
- Create: `src/alpha_studio/evaluation/alphalens_eval.py`

这两个模块是对 `pyfolio` / `alphalens` 的薄封装，主要逻辑是指标计算（可测）+ 第三方绘图调用（不强测）。

- ☐ **Step 1: Write the failing test (指标计算可测部分)**

追加到 `tests/backtest/test_engine.py`:

```
def test_performance_metrics():
    from alpha_studio.backtest import report
    rets = pd.Series([0.02, -0.01, 0.03, 0.01],
                     index=pd.date_range("2023-01-31", periods=4, freq="ME"))
    m = report.performance_metrics(rets)
    assert "annual_return" in m
    assert "sharpe" in m
    assert "max_drawdown" in m
    assert m["max_drawdown"] <= 0
```

- ☐ **Step 2: Run test to verify it fails**

Run: `python -m pytest tests/backtest/test_engine.py::test_performance_metrics -v` Expected:
FAIL — `ModuleNotFoundError: alpha_studio.backtest.report`

- ☐ **Step 3: Write minimal implementation**

`src/alpha_studio/backtest/report.py`:

```
import numpy as np
import pandas as pd
from loguru import logger

from alpha_studio.config import REPORTS_DIR

PERIODS_PER_YEAR = 12 # 月频

def performance_metrics(returns: pd.Series) -> dict:
    """从月频收益序列计算关键绩效指标。"""
    returns = returns.dropna()
    if returns.empty:
        return {"annual_return": np.nan, "sharpe": np.nan, "max_drawdown": np.nan}
    ann_return = (1 + returns).prod() ** (PERIODS_PER_YEAR / len(returns)) - 1
    ann_vol = returns.std(ddof=0) * np.sqrt(PERIODS_PER_YEAR)
    sharpe = ann_return / ann_vol if ann_vol > 0 else np.nan
    equity = (1 + returns).cumprod()
    drawdown = equity / equity.cummax() - 1
    return {
        "annual_return": float(ann_return),
        "annual_vol": float(ann_vol),
        "sharpe": float(sharpe),
        "max_drawdown": float(drawdown.min()),
        "win_rate": float((returns > 0).mean()),
    }
```

```
def save_tearsheet(returns: pd.Series, benchmark: pd.Series | None = None,
                  name: str = "backtest") -> None:
    """用 pyfolio 生成 tearsheet 图表并保存。绘图失败不影响主流程。"""
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        import pyfolio as pf

        pf.create_returns_tear_sheet(returns, benchmark_rets=benchmark,
return_fig=True)
        out = REPORTS_DIR / f"{name}_tearsheet.png"
        plt.savefig(out, bbox_inches="tight")
        plt.close("all")
        logger.info(f"saved tearsheet: {out}")
    except Exception as e: # noqa: BLE001
        logger.warning(f"tearsheet skipped: {e}")
```

src/alpha_studio/evaluation/__init__.py: (空文件)

src/alpha_studio/evaluation/alphalens_eval.py:

```
import pandas as pd
from loguru import logger

from alpha_studio.config import REPORTS_DIR

def evaluate_factor(factor: pd.Series, prices: pd.DataFrame, name: str,
                  quantiles: int = 5) -> dict:
    """用 Alphalens 评估单因子的 IC 与分组收益。返回 IC 摘要 dict。

    factor: MultiIndex(date, ticker) 因子值
    prices: MultiIndex(date, ticker) 含 'close'
    """
    try:
        import alphalens as al

        pricing = prices["close"].unstack("ticker").sort_index()
        factor_data = al.utils.get_clean_factor_and_forward_returns(
            factor, pricing, quantiles=quantiles, periods=(1,),
        )
        ic = al.performance.factor_information_coefficient(factor_data)
        summary = {"ic_mean": float(ic.mean().iloc[0]), "ic_std":
float(ic.std().iloc[0])}
        summary["ir"] = summary["ic_mean"] / summary["ic_std"] if
summary["ic_std"] else float("nan")
        logger.info(f"factor {name}: IC={summary['ic_mean']:.4f} IR=
{summary['ir']:.3f}")
        return summary
    except Exception as e: # noqa: BLE001
        logger.warning(f"alphalens eval failed for {name}: {e}")
```

```
return {"ic_mean": float("nan"), "ic_std": float("nan"), "ir":
float("nan")}
```

- ☐ **Step 4: Run test to verify it passes**

Run: `python -m pytest tests/backtest/test_engine.py::test_performance_metrics -v` Expected: PASS

- ☐ **Step 5: Commit**

```
git add -A
git commit -m "feat: performance metrics, pyfolio report, alphalens eval"
```

Task 11: CLI 串联

Files:

- Create: `src/alpha_studio/cli/__init__.py` (空)
- Create: `src/alpha_studio/cli/main.py`
- Create: `tests/test_cli.py`
- ☐ **Step 1: Write the failing test**

`tests/test_cli.py`:

```
from typer.testing import CliRunner
from alpha_studio.cli.main import app

runner = CliRunner()

def test_cli_help_lists_commands():
    result = runner.invoke(app, ["--help"])
    assert result.exit_code == 0
    for cmd in ["run-pipeline", "rank", "fetch-data", "eval-factors", "backtest"]:
        assert cmd in result.output

def test_rebalance_dates_are_month_end():
    from alpha_studio.cli.main import _rebalance_dates
    dates = _rebalance_dates("2023-01-01", "2023-04-15")
    # 月末日期
    assert all(d.is_month_end for d in dates)
    assert len(dates) == 3 # 1月、2月、3月末
```

- ☐ **Step 2: Run test to verify it fails**

Run: `python -m pytest tests/test_cli.py -v` Expected: FAIL — `ModuleNotFoundError: alpha_studio.cli.main`

- ❑ **Step 3: Write minimal implementation**

`src/alpha_studio/cli/__init__.py`: (空文件)

`src/alpha_studio/cli/main.py`:

```
import pandas as pd
import typer
from loguru import logger

from alpha_studio.config import REBALANCE_FREQ, TOP_N, TRANSACTION_COST
from alpha_studio.data.universe import get_sp500_tickers
from alpha_studio.data.prices import fetch_prices
from alpha_studio.data.fundamentals import fetch_fundamentals
from alpha_studio.factors.pipeline import build_from_raw
from alpha_studio.model.scorer import compute_forward_returns, walk_forward_score
from alpha_studio.backtest.engine import select_topn, run_backtest
from alpha_studio.backtest.report import performance_metrics, save_tearsheet

app = typer.Typer(help="股票价值投资 alpha 因子发掘系统")

def _rebalance_dates(start: str, end: str) -> pd.DatetimeIndex:
    return pd.date_range(start, end, freq=REBALANCE_FREQ)

def _monthly_prices_long(tickers, start, end) -> pd.DataFrame:
    prices = fetch_prices(tickers, start, end)
    monthly = prices["close"].unstack("ticker").resample(REBALANCE_FREQ).last()
    monthly_long = monthly.stack(future_stack=True).rename("close").to_frame()
    monthly_long.index = monthly_long.index.set_names(["date", "ticker"])
    return monthly_long

def _build_scores(tickers, start, end):
    rebal = _rebalance_dates(start, end)
    factors = build_from_raw(tickers, start, end, rebal)
    monthly_long = _monthly_prices_long(tickers, start, end)
    fwd = compute_forward_returns(monthly_long)
    scores = walk_forward_score(factors, fwd, min_train_dates=12)
    return scores, monthly_long

@app.command()
def fetch_data(start: str = "2018-01-01", end: str = "2024-12-31"):
    """仅更新数据缓存。"""
    tickers = get_sp500_tickers()
    fetch_prices(tickers, start, end)
    fetch_fundamentals(tickers)
    logger.info("data cache updated")
```

```

@app.command()
def eval_factors(start: str = "2018-01-01", end: str = "2024-12-31"):
    """跑 Alphalens 因子有效性诊断。"""
    from alpha_studio.evaluation.alphalens_eval import evaluate_factor
    tickers = get_sp500_tickers()
    rebal = _rebalance_dates(start, end)
    factors = build_from_raw(tickers, start, end, rebal)
    monthly_long = _monthly_prices_long(tickers, start, end)
    for col in factors.columns:
        evaluate_factor(factors[col].dropna(), monthly_long, col)

@app.command()
def rank(date: str = typer.Option(None, help="调仓月, 如 2024-06; 默认最近"),
        start: str = "2018-01-01", end: str = "2024-12-31"):
    """输出某月当期排名股票清单。"""
    tickers = get_sp500_tickers()
    scores, _ = _build_scores(tickers, start, end)
    if scores.empty:
        typer.echo("无打分结果")
        raise typer.Exit(1)
    target = scores.index.get_level_values("date").max() if not date \
        else pd.Timestamp(date) + pd.offsets.MonthEnd(0)
    day = scores.xs(target, level="date")["score"].nlargest(TOP_N)
    typer.echo(f"== {target.date()} Top {TOP_N} ==")
    for i, (tk, sc) in enumerate(day.items(), 1):
        typer.echo(f"{i:2d}. {tk:6s} {sc:+.4f}")

@app.command()
def backtest(start: str = "2018-01-01", end: str = "2024-12-31", topk: int =
TOP_N):
    """仅回测并输出绩效指标 + tearsheet。"""
    tickers = get_sp500_tickers()
    scores, monthly_long = _build_scores(tickers, start, end)
    weights = select_topn(scores, topk)
    result = run_backtest(weights, monthly_long, TRANSACTION_COST)
    metrics = performance_metrics(result["returns"])
    typer.echo("== 回测绩效 ==")
    for k, v in metrics.items():
        typer.echo(f"{k:16s}: {v:+.4f}")
    save_tearsheet(result["returns"], name="backtest")

@app.command()
def run_pipeline(start: str = "2018-01-01", end: str = "2024-12-31", topk: int =
TOP_N):
    """全流程: 拉数据→算因子→训练→回测→出报告。"""
    fetch_data(start, end)
    backtest(start, end, topk)
    rank(None, start, end)

```

```
if __name__ == "__main__":  
    app()
```

- ☐ **Step 4: Run test to verify it passes**

Run: `python -m pytest tests/test_cli.py -v` Expected: PASS (两个测试均通过)

- ☐ **Step 5: 全量测试 + README**

Run: `python -m pytest -q` Expected: 全部 PASS

创建 `README.md`:

```
# alpha-studio
```

巴菲特风格价值投资 alpha 因子发掘系统 (S&P 500, 月度调仓)。

```
## 安装
```

```
python -m pip install -e .
```

```
## 用法
```

```
sp fetch-data          # 更新数据缓存  
sp eval-factors        # 因子有效性诊断 (Alphalens)  
sp backtest --start 2018-01-01 --end 2024-12-31 --topk 25  
sp rank --date 2024-06 # 当期排名股票清单  
sp run-pipeline        # 全流程
```

```
## 已知局限
```

- 原型用 `yfinance` 基本面字段; 权威性弱于 SEC EDGAR (升级路径见 `spec`)
- 使用当前 S&P 500 成分股, 存在幸存者偏差

- ☐ **Step 6: Commit**

```
git add -A  
git commit -m "feat(cli): typer commands wiring + README"
```

Self-Review Notes

Spec coverage:

- 数据采集 (价格/基本面/universe) → Tasks 2–4 ✓
- 因子计算 + 防未来函数 + 标准化 → Tasks 5–7 ✓
- 因子评估 (Alphalens) → Task 10 ✓
- ML 多因子合成 (LightGBM walk-forward) → Task 8 ✓
- 月频回测 + 成本 + pyfolio 报告 → Tasks 9–10 ✓
- CLI 五个命令 → Task 11 ✓

- 错误处理（单股跳过、缓存、缺失剔除）→ Tasks 4, 7, 10 中体现 ✓
- 测试策略（因子手算、防未来函数、回测验证、mock 数据层）→ 各 Task 测试 ✓

类型一致性: `compute_forward_returns`、`walk_forward_score`、`select_topn`、`run_backtest` 的签名在定义任务与 CLI 调用处一致；因子矩阵统一 `MultiIndex(date, ticker)`；权重 Series 名为 `weight`；`_monthly_prices_long` 在 CLI 内复用，避免重复（DRY）。

已知简化（非占位符，明确决策）：EDGAR、历史成分股为 spec 第 12 节的未来升级项，原型不实现。